

```

    char vehno[10];
    int dist;
};
union info
{
    struct info1 a;
    struct info2 b;
};
struct emp
{
    char n[20];
    char grade[4];
    int age;
    union info f;
};
struct employee e;

```

## The *volatile* Qualifier

When we define variables in a function the compiler may optimize the code that uses the variable. That is, the compiler may compile the code in a manner that will run in the most efficient way possible. The compiler achieves this by using a CPU register to store the variable's value rather than storing it in stack.

However, if we declare the variable as *volatile*, then it serves as a warning to the compiler that it should not *optimize* the code containing this variable. In such a case whenever we use the variable its value would be loaded from memory into register, operations would be performed on it and the result would be written back to the memory location allocated for the variable.

We can declare a *volatile* variable as:

```
volatile int j;
```

Another place where we may want to declare a variable as *volatile* is when the variable is not within the control of the program and is likely to get altered from outside the program. For example, the variable

```
volatile float temperature;
```

might get modified through the digital thermometer attached to the computer.

## Summary

- The enumerated data type and the **typedef** declaration add to the clarity of the program.
- Typecasting makes the data type conversions for specific operations.
- When the information to be stored can be represented using a few bits of a byte we can use bit fields to pack more information in a byte.
- Every C function has an address that can be stored in a pointer to a function. Pointers to functions provide one more way to call functions.
- We can write a function that receives a variable number of arguments.
- Unions permit access to same memory locations in multiple ways.

## Exercise

[A] What will be the output of the following programs:

- ```
#include <stdio.h>
void main()
{
    enum status { pass, fail, atkt };
    enum status stud1, stud2, stud3;
    stud1 = pass;
```